CSD201 Library

Cheat Sheet

Java · List · Stack/Queue · Hash Table · Recursion · BST · Sorting · Graph

1 · MyList

package `list` · danh sách liên kết đơn, lưu `int`

Hàm	Cách dùng
Khởi tạo & thông tin	
`MyList()`	Tạo danh sách rỗng.
`boolean isEmpty()`	Kiểm tra rỗng.
`int size()`	Số node hiện có.
`void traversal()`	In các giá trị từ head đến tail.
Thêm	
`void addFirst(int x)`	Thêm `x` vào đầu danh sách.
`void addLast(int x)`	Thêm `x` vào cuối danh sách.
`boolean insertAt(int index, int x)`	Chèn tại `index` (từ `0` đến `size()`). → `false` nếu sai vị trí
`boolean insertBefore(int targetValue, int newValue)`	Chèn `newValue` trước node đầu tiên có giá trị `targetValue`.
`boolean insertAfter(int index, int x)`	Chèn `x` ngay sau node ở vị trí `index`.
Truy cập & cập nhật	
`Node get(int index)`	Lấy node tại `index`. → `null` nếu sai vị trí
`Node search(int x)`	Tìm node đầu tiên có giá trị `x`. → `null` nếu không thấy
`int getValue(int index)`	Lấy giá trị tại `index`. → ném `IndexOutOfBoundsException`
`boolean updateAt(int index, int newValue)`	Đổi giá trị của node tại `index`.
Xóa	
`boolean removeValue(int x)`	Xóa node đầu tiên có giá trị `x`.
`boolean removeAt(int index)`	Xóa node tại `index`.
`void remove(Node target)`	Xóa node `target` nếu thuộc danh sách.
`int removeAllLessThan(int x)`	Xóa mọi node có giá trị $<$ `x`. → số node đã xóa
`boolean removeAfter(int x)`	Xóa node **thứ hai** có giá trị $<$ `x`.
`void clear()`	Xóa toàn bộ, đưa danh sách về rỗng.
Min / Max & sắp xếp	
`int min()`	Giá trị nhỏ nhất. → ném `IllegalStateException` nếu rỗng
`int max()`	Giá trị lớn nhất. → ném `IllegalStateException` nếu rỗng
`boolean swapMinMax()`	Đổi chỗ giá trị min đầu tiên và max đầu tiên.
`void sort()`	Sắp xếp tăng dần (đổi giá trị, giữ liên kết). → $O(n^2)$
`boolean sortRange(int startIndex, int endIndex)`	Sắp xếp đoạn `[startIndex, endIndex]` (gồm cả hai đầu).
`void sortFirstThree()`	Sắp xếp 3 node đầu; ít hơn 3 node thì giữ nguyên.
`void reverse()`	Đảo ngược bằng cách đổi liên kết. → $O(n)$, bộ nhớ phụ $O(1)$
`boolean appendAll(MyList other)`	Nối bản sao giá trị của `other`. → `false` nếu `null` / chính nó

2 · MyStack & MyQueue

package `stackAndQueue` · lưu `Object`

Hàm	Cách dùng
MyStack (LIFO: vào sau, ra trước)	
`MyStack()`	Tạo stack rỗng.
`boolean isEmpty()`	Kiểm tra rỗng.
`void push(Object info)`	Đẩy `info` lên đỉnh stack.
`Object pop()`	Lấy và xóa phần tử đỉnh. → `null` nếu rỗng
`Object top()`	Xem phần tử đỉnh, không xóa. → `null` nếu rỗng
`void clear()`	Xóa toàn bộ phần tử.
MyQueue (FIFO: vào trước, ra trước)	
`MyQueue()`	Tạo queue rỗng.
`boolean isEmpty()`	Kiểm tra rỗng.
`void enqueue(Object info)`	Thêm `info` vào cuối hàng đợi.
`Object dequeue()`	Lấy và xóa phần tử đầu. → `null` nếu rỗng
`Object front()`	Xem phần tử đầu, không xóa. → `null` nếu rỗng
`void clear()`	Xóa toàn bộ phần tử.

3 · MyHashTable

package `hashTable` · nối chuỗi (chaining), $M = 10$ bucket

Hàm	Cách dùng
`MyHashTable()`	Tạo 10 bucket rỗng. → hash: `floorMod(x, 10)`
`void add(int x)`	Thêm `x` vào bucket tương ứng. → cho phép trùng
`void traversal()`	In các bucket không rỗng kèm chỉ số bucket.
`int search(int x)`	Tìm `x`. → trả về `x` nếu thấy, `-1` nếu không
`boolean remove(int x)`	Xóa **một** lần xuất hiện đầu tiên của `x`.

4 · Recursion

package `recursion` · toàn bộ là hàm `static`

Hàm	Cách dùng
`static int factorial(int n)`	Tính $n!$. → trả về `1` khi $n \leq 1$
`static void printRecursivePattern(int n)`	In n giảm dần rồi tăng dần. → `n=3` in `321123`
`static String generatePattern(int n)`	Sinh chuỗi theo công thức $f(n) = f(n-3) + n + f(n-2) + n$.
`static int calculateSum(int n)`	Tổng $1 + 2 + \cdots + n$. → `0` nếu $n \leq 0$
`static int findMin(int[] a, int n)`	Giá trị nhỏ nhất trong n phần tử đầu của `a`.
`static boolean isPalindrome(char[] characters, int n)`	Kiểm tra n ký tự đầu có đối xứng không.

5 · BSTree

package `tree` · cây nhị phân tìm kiếm, không nhận giá trị trùng

Hàm	Cách dùng
Node (`info`, `left`, `right`)	
`Node(int info)`	Tạo node không có con.
`Node(int info, Node left, Node right)`	Tạo node kèm hai con.
Cơ bản	
`BSTree()`	Tạo cây rỗng (`root = null`).
`boolean isEmpty()`	Kiểm tra cây rỗng.
`Node search(int key)`	Tìm node có giá trị `key`. → `null` nếu không thấy
`void insert(int value)`	Thêm `value`; bỏ qua nếu đã tồn tại.
`void visit(Node node)`	In giá trị của `node`.
`int height(Node node)`	Chiều cao cây con. → rỗng $= 0$, chỉ có gốc $= 1$
Duyệt cây (truyền vào `tree.root`)	
`void preOrder(Node node)`	Gốc → Trái → Phải.
`void inOrder(Node node)`	Trái → Gốc → Phải. → BST in ra tăng dần
`void postOrder(Node node)`	Trái → Phải → Gốc.
`void levelOrderTraversal(Node node)`	Duyệt theo từng tầng (dùng queue).
Xóa, cân bằng & xoay	
`void deleteByCopy(int value)`	Xóa; node 2 con được thay bằng **giá trị lớn nhất** bên trái.
`void deleteByMerging(int value)`	Xóa; cây con trái thế chỗ, cây con phải gắn vào node phải nhất bên trái.
`void balance()`	Cân bằng lại toàn bộ cây.
`Node rotateLeft(Node node)`	Xoay trái. → gốc mới của cây con
`Node rotateRight(Node node)`	Xoay phải. → gốc mới của cây con

6 · Sorting

package `sorting` · tất cả sắp xếp tăng dần, hàm của đối tượng

Hàm	Cách dùng
`void mergeSort(int[] array)`	Merge Sort. → $O(n \log n)$, bộ nhớ phụ $O(n)$
`void quickSort(int[] array)`	Quick Sort. → TB $O(n \log n)$, xấu nhất $O(n^2)$
`void selectionSort(int[] array)`	Selection Sort. → $O(n^2)$
`void insertionSort(int[] array)`	Insertion Sort. → $O(n^2)$
`void bubbleSort(int[] array)`	Bubble Sort, dừng sớm nếu đã sắp xếp. → $O(n^2)$
`void swap(int[] array, int firstIndex, int secondIndex)`	Đổi chỗ hai phần tử trong mảng.
`void saveToFile(int[] array, String fileName)`	Ghi mảng ra file, mỗi phần tử một dòng. → ném `IOException`

7 · Graph

package `graph` · ma trận kề · đỉnh đánh số từ 0 (0 → A, 1 → B, …)

Hàm	Cách dùng
Khởi tạo (`Graph.NO_EDGE = Integer.MAX_VALUE / 4`)	
`Graph(int vertexCount)`	Đồ thị vô hướng, không trọng số.
`Graph(int vertexCount, boolean weighted, boolean directed)`	Đồ thị tùy chọn có trọng số / có hướng.
`Graph(int[][] adjacencyMatrix)`	Từ ma trận kề vô hướng; `0` = không có cạnh.
`Graph(int[][] adjacencyMatrix, boolean weighted, boolean directed, int noEdgeValue)`	Từ ma trận kề, tự chỉ định giá trị biểu diễn **không có cạnh**.
Thông tin	
`int getVertexCount()`	Số đỉnh.
`boolean isWeighted()`	Đồ thị có trọng số không.
`boolean isDirected()`	Đồ thị có hướng không.
`void visit(int vertex)`	In nhãn đỉnh (A, B, …), không xuống dòng.
`void printMatrix()`	In ma trận kề. → `-` = không có cạnh, đường chéo = 0
Thêm / xóa cạnh	
`void addEdge(int from, int to)`	Thêm cạnh với trọng số mặc định `1`.
`void addEdge(int from, int to, int weight)`	Thêm cạnh có trọng số. → vô hướng: lưu hai chiều, bỏ self-loop
`void removeEdge(int from, int to)`	Xóa cạnh (vô hướng: xóa cả hai chiều).
Duyệt & tìm đường	
`void breadthFirstTraversal(int start)`	BFS (dùng queue); đồ thị không liên thông thì duyệt tiếp đỉnh chưa thăm.
`void depthFirstTraversal(int start)`	DFS (đệ quy); cũng duyệt các thành phần còn lại.
`void printConnectedComponents()`	In các thành phần liên thông. → chỉ đồ thị vô hướng
`void printPath(int start, int destination)`	In **một** đường đi bằng DFS. → không đảm bảo ngắn nhất
`void findShortestPath(int start, int destination)`	Dijkstra: in khoảng cách và đường đi. → trọng số ≥ 0
`void findEulerCircuit()`	Tìm chu trình Euler. → vô hướng, mọi đỉnh bậc chẵn, liên thông; không cạnh song song/self-loop